

AI

AI Certification Exam Assistant

TECHNICAL DOCUMENTATION & TEAM COLLABORATION GUIDE

Grounded certification practice from official knowledge to learner readiness

Architecture, implementation, quality controls, project runbook, and a clear five-member contribution model.

Python & FastAPI

OpenAI Responses API

FAISS RAG

SQLite

React 19 & Vite

DOCUMENT VERSION

1.0

PROJECT VERSION

0.3.0 MVP

PREPARED

28 August 2026

Table of contents

1. Executive overview
2. Goals, scope, and boundaries
3. System architecture
4. Certification data layer
5. RAG and vector retrieval
6. OpenAI AI Engine
7. Scoring and learner analytics
8. Backend API and application database
9. Frontend experience
10. Reliability and safety controls
11. Testing evidence
12. Five-member work allocation
13. Collaboration and handoff model
14. Local setup and runbook
15. Quick team demo sharing
16. Limitations and next roadmap
17. Appendix A: folder map
18. Appendix B: API reference
19. Appendix C: environment configuration

Team-name placeholders: the responsibility section uses Member 1–5. Replace each placeholder with the real contributor name before submission or presentation.

What the project does

The AI Certification Exam Assistant is a grounded learning system that converts repository-controlled certification knowledge into new practice-only multiple-choice questions, evaluates learner answers, detects weak areas, recommends official resources, builds a study plan, and calculates readiness through transparent rules.

5

Certification JSON source files

38

Prepared semantic RAG chunks

4

Certification-scoped FAISS indexes

30

Passing Python tests

Official JSON

Certification source of truth

RAG

Clean, chunk, embed, retrieve

OpenAI

Structured grounded questions

FastAPI

Auth, orchestration, persistence

React

Assessment and learner insights

Core outcome: official certification knowledge → semantic retrieval → grounded practice → deterministic evaluation → personalized preparation.

Current production-candidate corpus

AWS Certified AI Practitioner (AIF-C01)

contains five domains and fourteen detailed, source-linked topic/task chunks. It is the only corpus currently enabled for live question generation.

Reliability-first behavior

When retrieved content is incomplete, the system returns `insufficient_context` or blocks generation. It does not fill knowledge gaps with internet content or unsupported model knowledge.

Goals, users, and boundaries

Primary goals

- ✓ Generate new practice questions from retrieved official content.
- ✓ Keep certification retrieval isolated before vector search.
- ✓ Return machine-readable, validated question JSON.
- ✓ Score MCQs without using an LLM.
- ✓ Turn results into weak areas, official recommendations, study plans, and readiness.
- ✓ Expose the proven AI foundation through a modular API and responsive learner interface.

Explicit content policy

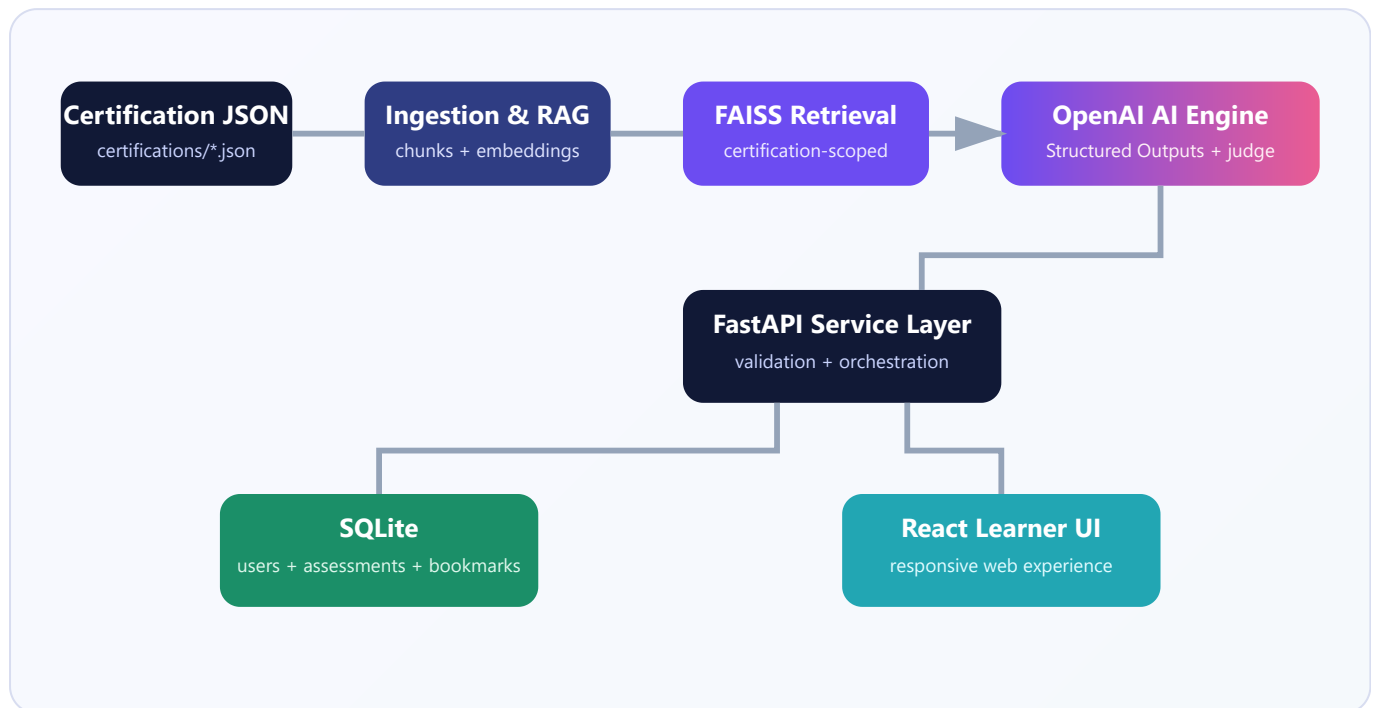
- Certification JSON files are the source of truth.
- No leaked or real exam questions are used.
- No random web content is added at runtime.
- Generated items are always marked `practice_only=true`.
- Source chunk IDs and official URLs must be preserved.
- Unsupported certification corpora remain disabled.

Implemented MVP capabilities

Area	Implemented capability
Identity	Registration, login, signed bearer token, user-isolated records, Argon2 password hashing.
Knowledge	Dynamic JSON discovery, conservative cleaning, topic-aware chunking, OpenAI embeddings, FAISS persistence.
Assessment	Difficulty and count configuration, grounded MCQ generation, hidden answer keys until submission.
Analytics	Deterministic scoring, domain/topic performance, weak areas, recommendations, study plan, readiness.
Learner UI	Dashboard, assessment, results, sources, weak areas, bookmarks, history, settings, help, responsive navigation.

Deferred: production deployment, managed database migrations, production secrets, rate limiting, token revocation, cloud monitoring, backups, and full browser end-to-end automation remain future work.

End-to-end system design



Runtime request sequence

1. A signed-in learner selects certification, difficulty, question count, and study-plan days.
2. FastAPI checks that the selected corpus is detailed enough for question generation.
3. The retriever selects the certification-specific FAISS index **before** embedding and searching the query.
4. Retrieved chunks are sent to the AI Engine, which generates a strict Pydantic question batch.
5. Deterministic and semantic grounding checks accept the batch or reject/regenerate it.
6. FastAPI stores the assessment while keeping correct answers and explanations server-side.
7. After submission, Python computes the score and all personalized outputs; SQLite stores the completed result.
8. The React client renders results, sources, weak topics, plan, readiness, bookmarks, and history.

Certification knowledge as the source of truth

The loader dynamically discovers every `certifications/*.json` file. A certification contains identity metadata, domains, topics/tasks, subtopics, content, domain weight, and official source metadata. Source content is retained rather than silently rewritten.

Certification	Domains	Topics	Chunks	Detailed	Generation status
AWS Certified AI Practitioner	5	14	14	14	Enabled
Microsoft Azure AI Fundamentals	2	7	7	0	Blocked by quality guard
IBM AI Engineering Professional Certificate	4	10	10	0	Blocked by quality guard
DeepLearning.AI Programs	2	7	7	0	Blocked by quality guard
Google Generative AI Leader	4	0	0	0	Requires structured detailed topics

Metadata preserved through the pipeline

Certification

- `certification_id`
- `certification`
- `provider`

Knowledge structure

- `domain`
- `domain_weight`
- `topic`
- `subtopics`

Provenance

- `source_title`
- `source_url`
- `source_type`
- `source_urls`

Cleaning rules

- Remove empty or unusable records.
- Normalize redundant whitespace and line endings.
- Remove malformed control and replacement characters.
- Deduplicate exact content records and repeated sources.
- Preserve factual meaning, certification identity, topic structure, and official URLs.

RAG and vector database pipeline

Topic-aware chunking

One certification topic/task remains one semantic unit. Only unusually long topic content is split, using paragraph or sentence boundaries. The default maximum is approximately 3,500 characters with a small 250-character overlap. Each chunk receives a stable SHA-256-derived ID and retains all metadata.

Why this fits certification knowledge: exam guides are already organized into domains, tasks, and subtopics. Preserving those units avoids mixing unrelated objectives and gives each generated question a precise topic and source trail.

Embedding and index design

Embedding provider	OpenAI Python SDK	Model	text-embedding-3-small
Vector dimension	1,536	Metric	Cosine similarity through L2-normalized inner product
FAISS index	IndexFlatIP	Persistence	Global index, metadata manifest, and per-certification indexes

Pre-search certification isolation

```
retrieve_context(query, certification_id="aif-c01", top_k=5)
↓
Resolve vector_store/by_certification/aif-c01.faiss
↓
Embed the query with the same stored embedding model
↓
Search only AWS positions and return text, domain, topic, source, score
```

An unknown certification ID is rejected before the query embedding call. An AWS selection therefore cannot retrieve Microsoft, IBM, Google, or DeepLearning.AI chunks.

Persisted RAG artifacts

- data/rag_chunks.json — cleaned, validated semantic chunks.
- vector_store/index.faiss — global inspection index.
- vector_store/metadata.json — model, dimensions, chunk metadata, index mappings.
- vector_store/by_certification/*.faiss — certification-scoped search indexes.

OpenAI AI Engine

The AI Engine uses the OpenAI Responses API and Pydantic Structured Outputs. The generation model is configurable with `OPENAI_GENERATION_MODEL` and currently defaults to `gpt-5-mini`.

OpenAI-assisted functions

- Grounded practice-question generation.
- Closed-book semantic grounding review.
- Optional explanation wording for weak areas.
- Optional recommendation wording.
- Optional study-plan wording.
- Optional readiness explanation.

Deterministic functions

- Answer and assessment scoring.
- Domain and topic accuracy.
- Weak-topic thresholds and priority.
- Allowed source URL enforcement.
- Study-plan topic/day structure.
- Readiness score and classification.

Question output contract

```
{
  "question": "...",
  "options": {"A": "...", "B": "...", "C": "...", "D": "..."},
  "correct_answer": "A",
  "explanation": "...",
  "difficulty": "medium",
  "certification": "AWS Certified AI Practitioner",
  "domain": "...",
  "topic": "...",
  "source_chunk_ids": ["chunk_..."],
  "source_urls": ["https://official-source.example/..."],
  "practice_only": true
}
```

Generation acceptance gates

1. Context must contain detailed explanatory content and an official URL.
2. Every context record must match the requested certification.
3. Exactly four distinct options and the requested question count are required.
4. Domain, topic, chunk IDs, and URLs must exist in cited retrieved records.
5. Every option, the answer, and explanation must pass deterministic grounding checks.
6. A separate model call performs strict semantic entailment review.
7. A failed batch may be regenerated with validator feedback, up to three attempts.
8. If a reliable batch cannot be produced, the API returns an error instead of unsupported questions.

Scoring, weak areas, recommendations, study plan, and readiness

Output	How it is calculated	LLM authority
Assessment score	Known answer key compared with submitted A/B/C/D selections. Missing answers are incorrect.	None
Domain/topic performance	Total, correct, incorrect, and accuracy grouped in Python.	None
Weak areas	Topics below 80% require review; below 50% is high priority by default.	May explain, cannot change scores
Recommendations	Weak topics mapped only to official URLs present in supplied resources.	May improve wording within allowlists
Study plan	High-priority and lowest-accuracy topics ordered and cycled across available days.	Cannot change topics, days, priorities, or URLs
Readiness	≥80 Ready; 60–79 Nearly Ready; <60 Needs Improvement. Thresholds are configurable.	May explain, cannot classify

Submitted answers
A/B/C/D

Python score
Question + topic + domain

Weak areas
Threshold priorities

Action plan
Official recommendations

Readiness
Transparent classification

FastAPI backend and SQLite database

FastAPI is a thin orchestration and persistence layer over the tested `certification_assistant` package. AI rules are not duplicated in route handlers. Pydantic request/response schemas reject unexpected fields and invalid ranges.

Backend responsibilities

API
Authentication, certification catalog, assessment generation/submission, result views, dashboard, bookmarks, and history.

Security
Argon2 hashes, JWT HS256 bearer tokens, active-user validation, CORS configuration, and ownership filters.

Orchestration
Corpus readiness guard, RAG call, AI generation, deterministic analytics, persistence, and public answer-key filtering.

Application database schema

Table	Important fields	Relationship and purpose
<code>users</code>	id, email, full_name, hashed_password, is_active, created_at	Unique email identity. One user owns many assessments and bookmarks.
<code>assessments</code>	id, user_id, certification, difficulty, questions_json, answers_json, result_json, weak_areas_json, study_plan_json, readiness_json, status	Stores the complete learner workflow. Foreign key to users with cascade delete.
<code>bookmarks</code>	id, user_id, question_key, certification_id, question_json, note	User-owned saved practice questions. Unique per user/question pair.

Two different databases serve different purposes: FAISS stores vector knowledge for retrieval; SQLite stores accounts and learner activity. They should not be presented as the same database component.

Privacy of assessment answers

Before submission, the public assessment schema excludes `correct_answer` and `explanation`. These fields remain in the server-side assessment record and appear only in the post-submission question review.

React frontend and API integration

The learner interface uses React 19, Vite, native Fetch, Lucide icons, semantic form controls, theme persistence, hash-based page state, loading and error states, and a shared API client with bearer-token handling.

Page	Main learner capability
Authentication	Register or sign in; show/hide password; clear inline API feedback.
Dashboard	Readiness summary, score metrics, certification readiness, and assessment configuration.
Assessment	Question navigation, progress, answer selection, bookmarking, and submission.
Results	Score, readiness, domain/topic results, answer review, explanations, citations, and recommendations.
Weak Areas	Priority-filtered topics derived from deterministic accuracy.
Study Plan	Day-by-day activities and links to official resources.
History & Bookmarks	Persistent attempts, result reopening, saved practice questions, and removal.
Settings & Help	Theme choice, account summary, and project-accurate guidance.

Responsive behavior

- Viewport metadata is configured for mobile browsers.
- The fixed desktop sidebar becomes an accessible drawer below 900px.
- Dashboard, assessment, results, settings, and study-plan grids collapse at page-specific breakpoints.
- Buttons and key controls use touch-friendly sizing.
- Horizontal overflow is contained or made intentionally scrollable for dense history/filter views.
- Reduced-motion preferences disable nonessential transitions.

Single-URL demo proxy

The frontend calls relative `/api`. Vite proxies those requests to `http://127.0.0.1:8000`, enabling a VS Code public tunnel on port 5173 without separately exposing backend port 8000.

Reliability, grounding, and safety controls

Knowledge controls

- Repository JSON is the only knowledge source.
- Certification-scoped search occurs before retrieval.
- Embedding model compatibility is checked against the index manifest.
- Generation is disabled for topic-label-only corpora.
- Official URLs are allowlisted from retrieved content.

Question controls

- Strict Pydantic Structured Output.
- Exact four-option contract.
- Practice-only flag enforcement.
- Duplicate question detection.
- Option, answer, and explanation grounding.
- Independent semantic audit and safe failure.

Application controls

- Argon2 password hashes.
- Signed expiring bearer tokens.
- Assessment ownership checked by user ID.
- Correct answers hidden pre-submission.
- Environment-based secrets and thresholds.

Transparency controls

- Source chunk IDs and URLs preserved.
- Deterministic scores and readiness.
- Corpus readiness warning displayed.
- Results can be reviewed question by question.
- Unsupported generation returns an explicit error.

Important quality statement: passing automated checks is a quality gate, not a guarantee that every future model output is correct. Grounding metrics, rejected batches, and user feedback should continue to be monitored.

Testing evidence and acceptance criteria

30

Python tests passed on 28 Aug 2026

0

Python test failures

Pass

Frontend Oxlint

Pass

Vite production build

Automated test coverage

Test area	What is verified
Ingestion and chunks	Dynamic loading, metadata retention, cleaning, semantic chunking, stable outputs.
Retrieval	Certification filtering before search, expected result structure, unknown-ID rejection.
AI contracts	Structured question schema, insufficient-context guard, source and option grounding, unsupported output rejection.
Evaluation	Correct/incorrect totals, percentage, domain/topic breakdown, missing-answer behavior.
Personalization	Weak-area priority, official recommendation URLs, study-plan constraints, readiness boundaries.
Backend integration	Registration/login, catalog, assessment lifecycle, answer-key privacy, ownership, dashboard, history, bookmarks.
Frontend quality	Static lint rules and successful optimized production build.

Verification commands

```
# From project root
python -m pytest backend/tests tests -q

# From frontend/
npm run lint
npm run build
```

The verified environment reported 30 passed tests and one third-party deprecation warning from the FastAPI/Starlette test client dependency path; it did not represent a test failure.

Five-member work allocation

The ownership model below gives every contributor a distinct technical responsibility, visible folder ownership, measurable deliverables, and a defined handoff to the next subsystem.

1

Data Collection & Certification Content

Official knowledge acquisition, structure, provenance, and content readiness

Name: _____

Primary work

- Collect and structure certification knowledge only from approved official sources.
- Maintain certification identity, provider, domains, weights, topics, subtopics, detailed content, and source metadata.
- Paraphrase faithfully without adding leaked questions or unsupported facts.
- Identify incomplete certifications and document content gaps.
- Validate JSON syntax and content-level completeness before ingestion.

Deliverables

Source-of-truth JSON files, source register, data-quality checklist, corpus status notes, and update history.

Folder ownership

certifications/

certifications/aws_ai_practitioner.json

certifications/*.json

Shared review:

README.md

data section and notebook source examples.

Handoff

Provides valid hierarchical JSON to Member 2.
Accepts content-gap reports from Members 2 and 3.

2

RAG Pipeline & Vector Database

Ingestion, cleaning, chunking, embeddings, FAISS persistence, and retrieval

Name: _____

Primary work

- Load all certification JSON files dynamically.
- Normalize content without changing factual meaning.
- Create metadata-rich topic-aware chunks and stable chunk IDs.
- Generate OpenAI embeddings and build global plus per-certification FAISS indexes.
- Implement certification-filtered retrieval and similarity scoring.
- Maintain the corpus quality report and retrieval tests.

Deliverables

Prepared chunks, vector-store manifest, FAISS indexes, retriever API, retrieval evaluation, and reproducible preparation commands.

Folder ownership

```
src/certification_assistant/ingestion /
src/certification_assistant/rag/
data/rag_chunks.json
vector_store/
notebooks/01_rag_pipeline.ipynb
scripts/create_notebook.py
```

Handoff

Supplies typed retrieved chunks to Member 3 and corpus-readiness metadata to Member 4.

3

OpenAI & AI Engine

Structured question generation, grounding gates, and learning intelligence

Name: _____

Primary work

- Design the retrieved-context-only question prompt.
- Use the Responses API with Pydantic Structured Outputs.
- Validate certification, domain, topic, source IDs, URLs, options, answer, and explanation.
- Run semantic grounding review and reject unsupported batches.
- Implement deterministic scoring, weak-area detection, official recommendations, study plans, and readiness.
- Define strict internal schemas and AI failure behavior.

Deliverables

AI Engine functions, Pydantic contracts, grounding validator, deterministic analytics, and AI-specific tests.

Folder ownership

```
src/certification_assistant/ai/
src/certification_assistant/assessment/
src/certification_assistant/schemas.py
scripts/run_live_pipeline.py
tests/test_ai_engine.py
```

Handoff

Exposes stable Python service functions and schemas to Member 4. Reports content insufficiency to Members 1 and 2.

Primary work

- Create FastAPI routes and Pydantic API request/response models.
- Implement registration, login, password hashing, signed tokens, and current-user dependency.
- Design SQLAlchemy users, assessments, and bookmarks models.
- Persist complete assessment analytics and isolate all user-owned records.
- Hide answer keys until submission and map AI failures to safe API responses.
- Build API integration tests and maintain environment-backed configuration.

Deliverables

Runnable API, OpenAPI documentation, local SQLite schema, auth flow, assessment lifecycle, persistence, and backend test suite.

Folder ownership

`backend/app/api/`
`backend/app/core/`
`backend/app/models/`
`backend/app/schemas/`
`backend/app/services/`
`backend/tests/`
`data/assistant.db` (runtime)

Handoff

Publishes the documented

`/api`

contract consumed by Member 5. Calls Member 2 and 3 modules rather than duplicating them.

Primary work

- Build registration/login, dashboard, assessment, results, weak areas, plan, history, bookmarks, settings, and help pages.
- Create reusable responsive layout and theme styling.
- Integrate the FastAPI contract through one shared API client.
- Store bearer tokens and theme preferences in the browser.
- Implement loading, timeout, empty, error, progress, and success states.
- Maintain mobile navigation, accessibility labels, source links, and frontend verification.

Deliverables

React/Vite web application, responsive CSS, API integration, single-URL demo proxy, lint-clean source, and production build.

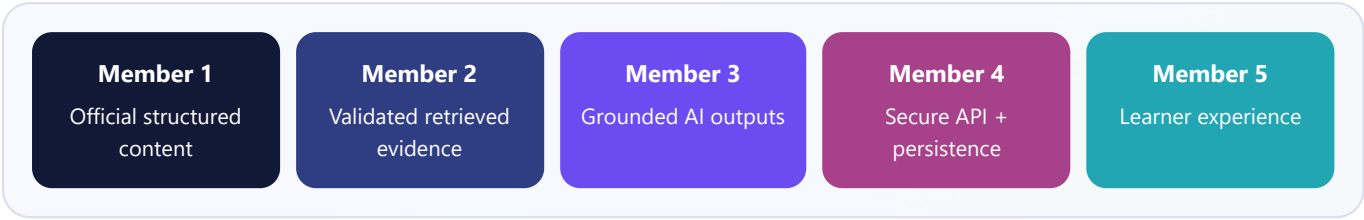
Folder ownership

```
frontend/src/components/  
frontend/src/services/api.js  
frontend/src/App.jsx  
frontend/src/*.css  
frontend/public/  
frontend/vite.config.js
```

Handoff

Consumes Member 4's API schemas and reports integration errors. Uses sources and analytics supplied by Members 2 and 3 through the backend.

How the five members collaborate



Responsibility matrix

Work item	Accountable owner	Required reviewers	Acceptance evidence
Certification source update	Member 1	Members 2 and 3	Valid JSON, official URLs, detailed content, no unsupported claims
Chunk/index rebuild	Member 2	Members 1 and 3	Preparation report, ready manifest, scoped retrieval tests
Prompt/schema change	Member 3	Members 2 and 4	Structured output tests, grounding tests, live smoke example
API/database change	Member 4	Members 3 and 5	OpenAPI contract, migration/schema note, API tests
Frontend workflow change	Member 5	Member 4	Lint, build, responsive and error-state review
End-to-end release candidate	Shared team	All members	Python suite, frontend checks, live grounded assessment, manual demo checklist

Recommended collaboration workflow

1. Create a short issue with owner, inputs, expected output, and acceptance tests.
2. Change only the owning subsystem unless an integration contract also requires an agreed update.
3. Keep JSON schemas, Pydantic schemas, and frontend API expectations synchronized.
4. Run local subsystem tests before requesting review.
5. Use a small integration fixture for routine work; use live OpenAI only for defined smoke tests.
6. Document corpus changes and vector-index rebuilds together.
7. Do not commit `.env`, API keys, local SQLite records, or temporary demo credentials.

Local setup and runbook

Prerequisites

Python 3.11+, Node.js/npm, an OpenAI API key, and internet access for embedding and generation calls.

Install Python dependencies

```
cd C:\Users\jamaa\Documents\ai-certification-assistant
python -m venv .env
.venv\Scripts\Activate.ps1
python -m pip install -e ".[notebook,test]"
Copy-Item .env.example .env
```

Prepare and embed certification data

```
cert-rag prepare
cert-rag embed
```

Run backend and frontend

Terminal 1 — FastAPI

```
python -m uvicorn backend.app.main:app \
  --reload --host 127.0.0.1 --port 8000
```

Health: <http://127.0.0.1:8000/health>
Docs: <http://127.0.0.1:8000/docs>

Terminal 2 — React

```
cd frontend
npm install
npm run dev
```

Application: <http://127.0.0.1:5173>

Live AI smoke test

```
python scripts\run_live_pipeline.py --questions 3 --days 5
```

Generation time: assessment creation includes retrieval, structured question generation, deterministic validation, a second semantic-review call, and possible regeneration. A grounded batch can therefore take noticeably longer than a normal API request.

Quick team demo through one public URL

For a temporary team demonstration, keep FastAPI on local port 8000 and expose only Vite port 5173. The Vite proxy forwards browser requests from `/api` to the local backend.

1. Start backend on `127.0.0.1:8000`.
2. Start frontend on `127.0.0.1:5173`.
3. In VS Code, open the **Ports** view.
4. Forward port **5173**.
5. Set Port Visibility to **Public**.
6. Copy the generated `https://...devtunnels.ms` address and share it with the team.
7. Stop forwarding after the demonstration.

What every tester can do

- ✓ Create a separate learner account.
- ✓ Generate an AWS AIF-C01 practice assessment.
- ✓ Submit answers and receive an isolated result.
- ✓ Review sources, weak areas, recommendations, study plan, readiness, history, and bookmarks.
- ✓ Open the same public HTTPS URL from desktop, tablet, Android, or iPhone.

Demo limitations: the host laptop and both servers must stay running; anyone with the public link can reach the application; all OpenAI usage is charged to the configured project key; simultaneous live generations can be slower or encounter provider rate limits. This tunnel is not a production deployment.

Current limitations and recommended roadmap

Priority	Next improvement	Reason	Likely owners
1	Background assessment jobs with progress states	Prevent long synchronous requests and improve mobile/demo experience.	Members 4 and 5
2	Regenerate only failed questions	Reduce latency and OpenAI cost while keeping strict validation.	Member 3
3	Validated question pool and duplicate tracking	Serve common assessments faster and avoid repeated learner items.	Members 3 and 4
4	Expand official detailed corpora	Enable Microsoft, IBM, Google, and DeepLearning.AI without lowering grounding quality.	Members 1 and 2
5	Domain-weighted and adaptive assessments	Align question coverage with official blueprints and learner performance.	Members 2, 3, and 5
6	Production security and deployment	Managed secrets, HTTPS, rate limiting, backups, observability, migrations, and scalable persistence.	Member 4 with team review
7	Browser E2E automation	Verify complete flows across desktop and mobile breakpoints.	Members 4 and 5

Recommended release order: instrument generation timing → add background progress → retry only failed questions → expand content → deploy with production security. This preserves the project's reliability-first design.

Folder and module map

```
ai-certification-assistant/
├── certifications/                # Member 1: official source-of-truth JSON
├── src/certification_assistant/
│   ├── ingestion/                # Member 2: loading and cleaning
│   ├── rag/                      # Member 2: chunks, embeddings, FAISS, retrieval
│   ├── ai/                       # Member 3: OpenAI generation and grounding
│   ├── assessment/               # Member 3: deterministic learning analytics
│   └── schemas.py                 # Member 3: internal typed contracts
├── backend/app/
│   ├── api/                      # Member 4: FastAPI routes
│   ├── core/                    # Member 4: config, DB, security
│   ├── models/                  # Member 4: SQLAlchemy entities
│   ├── schemas/                 # Member 4: public API contracts
│   └── services/                 # Member 4: orchestration
├── frontend/
│   ├── src/components/           # Member 5: learner pages and responsive CSS
│   ├── src/services/api.js       # Member 5: API client
│   └── vite.config.js            # Member 5: development and demo proxy
├── data/
│   ├── rag_chunks.json           # Member 2: generated retrieval artifact
│   └── assistant.db              # Member 4: local runtime database
├── vector_store/                 # Member 2: FAISS and manifest artifacts
├── notebooks/                    # Member 2/3: evaluation and demonstration
├── scripts/                      # Member 2/3: reproducible pipelines
├── tests/                        # Member 2/3: AI and RAG tests
└── backend/tests/                # Member 4: API integration tests
```

API route reference

Method and route	Authentication	Purpose
GET /health	No	Service health check.
POST /api/auth/register	No	Create user and return bearer token.
POST /api/auth/login	No	Verify password and return bearer token.
GET /api/auth/me	Yes	Restore the current user session.
GET /api/certifications	No	List catalog, chunk counts, and generation readiness.
GET /api/dashboard	Yes	User-isolated score and activity summary.
POST /api/assessments	Yes	Retrieve evidence, generate questions, and persist assessment.
GET /api/assessments/{id}	Yes	Read an owned in-progress assessment without answer key.
POST /api/assessments/{id}/submit	Yes	Score and persist full learner analytics.
GET /api/assessments/history	Yes	List owned assessment attempts.
GET /api/results/latest	Yes	Return latest completed result and question review.
GET /api/results/{id}	Yes	Return one owned completed result.
GET /api/weak-areas/latest	Yes	Latest weak-area analysis.
GET /api/recommendations/latest	Yes	Latest official-source recommendations.
GET /api/study-plan/latest	Yes	Latest personalized study plan.
GET /api/readiness/latest	Yes	Latest deterministic readiness result.
GET /api/bookmarks	Yes	List current user's bookmarks.
POST /api/bookmarks	Yes	Save an owned assessment question.
DELETE /api/bookmarks/{id}	Yes	Delete an owned bookmark.

Environment configuration

Variable	Purpose	Example
OPENAI_API_KEY	Secret used for embeddings and live generation.	your_real_key_here
OPENAI_EMBEDDING_MODEL	Embedding model used to build and query FAISS.	text-embedding-3-small
OPENAI_GENERATION_MODEL	Model used by Structured Outputs and semantic review.	gpt-5-mini
READINESS_READY_THRESHOLD	Minimum score for Ready.	80
READINESS_NEARLY_READY_THRESHOLD	Minimum score for Nearly Ready.	60
APP_ENVIRONMENT	Application mode and production-secret enforcement.	development
APP_SECRET_KEY	JWT signature secret.	replace-with-long-random-secret
ACCESS_TOKEN_EXPIRE_MINUTES	Bearer-token lifetime.	1440
DATABASE_URL	SQLAlchemy application database connection.	sqlite:///./data/assistant.db
CORS_ORIGINS	Allowed direct frontend origins when not using same-origin proxy.	http://127.0.0.1:5173

Secret handling: place real values in the root `.env` , never in `.env.example` . Do not commit, screenshot, email, or include real API keys in this document.